

Sistema de reservas de salas de estudio — Informe técnico

Autor (ficticio): Estudiante A — Grupo 1 **Materia:** Ingeniería de Software

1. Definición del problema y requisitos

La biblioteca central gestiona la reserva de sus 12 salas de estudio grupales con una hoja de cálculo compartida y un cuaderno físico en recepción. Esto genera reservas duplicadas sobre la misma sala y franja horaria, salas reservadas que quedan vacías porque nadie cancela, y no hay forma de saber la ocupación histórica para planificar. El proyecto propone un sistema web que centralice la reserva y libere automáticamente las salas no confirmadas.

Alcance. El sistema cubre la reserva, confirmación y cancelación de salas por parte de estudiantes, y la consulta de ocupación por parte del personal. Queda fuera del alcance la gestión de préstamos de libros y el control de acceso físico a las salas.

Requisitos funcionales (verificables):

- RF1: Un estudiante autenticado puede reservar una sala libre indicando fecha y franja de 1 hora; el sistema rechaza la reserva si la franja ya está ocupada.
- RF2: El sistema marca como disponible una reserva que no se confirma dentro de los 15 minutos posteriores al inicio de la franja.
- RF3: Un estudiante puede cancelar únicamente sus propias reservas futuras.
- RF4: El personal puede consultar la ocupación de cada sala por día y exportarla en CSV.

Requisitos no funcionales:

- RNF1 (rendimiento): la consulta de disponibilidad responde en menos de 500 ms con hasta 200 reservas activas.
- RNF2 (disponibilidad): el sistema opera en el horario de la biblioteca (07:00–21:00) con una disponibilidad objetivo del 99%.
- RNF3 (seguridad): cada estudiante solo accede a sus propias reservas; el personal tiene un rol distinto con acceso de solo lectura a la ocupación agregada.

2. Diseño de la arquitectura

El sistema sigue una arquitectura en tres capas con un frontend desacoplado:

```
[ SPA Web ] --HTTP/JSON--> [ API REST ] --> [ Servicio de Reservas ] --> [ PostgreSQL ]
                                     |
                                     +--> [ Job de liberación (cron 5 min) ]
```

- **SPA Web:** interfaz de reserva y consulta; consume la API por HTTP/JSON.
- **API REST:** autenticación, validación de entrada y enrutamiento a los servicios.

- **Servicio de Reservas:** contiene la lógica de negocio (evitar solapamientos, reglas de cancelación). Es el único componente que escribe en la tabla de reservas.
- **Job de liberación:** tarea programada que cada 5 minutos detecta reservas no confirmadas vencidas (RF2) y las libera.
- **PostgreSQL:** persistencia de usuarios, salas y reservas.

Flujo de una reserva: la SPA envía la solicitud → la API valida el token y el formato → el Servicio de Reservas comprueba en una transacción que la franja siga libre y la inserta con una restricción de unicidad (`sala_id`, `fecha`, `franja`) que garantiza RF1 incluso ante solicitudes concurrentes → responde la confirmación a la SPA.

3. Justificación de decisiones técnicas

- **PostgreSQL sobre una hoja de cálculo o NoSQL:** el problema central son las reservas duplicadas, que son una condición de carrera. Una restricción de unicidad a nivel de base de datos relacional resuelve RF1 de forma atómica; una alternativa como MongoDB exigiría manejar la unicidad en la aplicación, con más riesgo de solapamiento bajo concurrencia. Por eso se eligió una base relacional.
- **Job programado para RF2 en lugar de validación al momento de consultar:** liberar las salas con una tarea cada 5 minutos mantiene los datos consistentes para todos los lectores sin depender de que alguien consulte. Se descartó liberar "perezosamente" en cada consulta porque dejaría la ocupación incorrecta entre consultas, afectando RF4.
- **SPA desacoplada de la API:** permite que el personal use la misma API para exportar CSV (RF4) y habilita una futura app móvil sin reescribir la lógica. El trade-off es mayor complejidad inicial de despliegue, aceptable para el alcance previsto.

4. Fundamentación teórica y referencias

El diseño en capas con una única vía de escritura sigue el principio de separación de responsabilidades descrito por Martin (2017), que reduce el acoplamiento entre la lógica de negocio y la presentación. La decisión de garantizar la unicidad en la base de datos en lugar de en la aplicación se apoya en el análisis de anomalías de concurrencia de Kleppmann (2017), donde se muestra que las restricciones de integridad del motor son más fiables que las verificaciones a nivel de aplicación bajo transacciones concurrentes. Para la definición verificable de requisitos no funcionales se siguió la clasificación de Sommerville (2016).

Referencias

- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
- Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
- Sommerville, I. (2016). *Software Engineering* (10.^a ed.). Pearson.

5. Conclusión

El sistema reemplaza un proceso manual propenso a errores por uno que garantiza la no duplicación a nivel de datos, libera salas ociosas automáticamente y habilita la planificación a partir de la ocupación histórica, cumpliendo los requisitos definidos.